Block 2: Database Day 1 - JDBC & Object Mapping

March 2026

Module overview

d

Day 1

JDBC & Object Mapping

F

Day 2

Transactions & Concurrency

Day 3

Abstraction Layers & Hibernate

o

Day 4

Entity Relations & Fetch Strategies

Day 5

ORM Caching & Performance

Ice breaker

- What is one thing from last block that stayed with you?
- If aliens landed on earth tomorrow and offered to take you home with them, would you go?
- What do you hope to learn about databases?

Time planning

r Starting time: 9.30

W Breaks: …

End time: 16.30

Learning goals

- Understand JDBC and how Java communicates with databases
- Write safe database queries using PreparedStatements
- Map database results to Java objects
- Apply the DAO pattern for clean data access
- Use DataSources for connection management

0

Section 1

Introduction: What is JDBC?

The problem

Z Java lives in the JVM

d Your data lives in a database

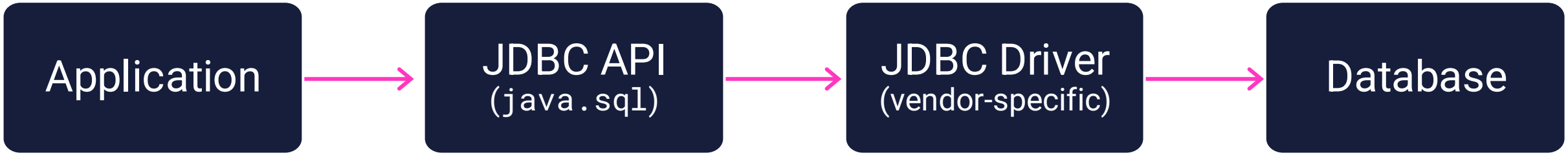
S How do they talk to each other?

The solution: JDBC

What is JDBC?

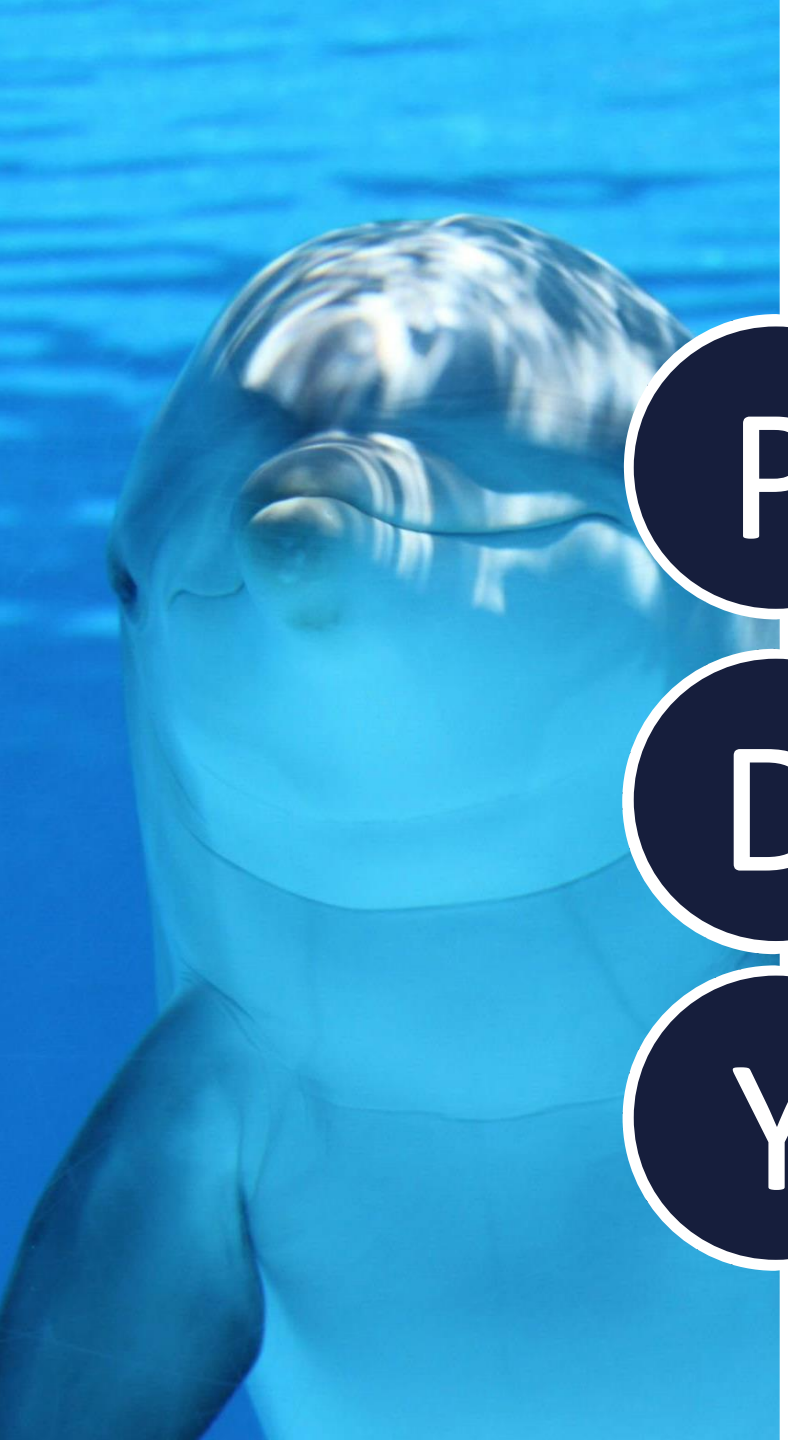
- Java Database Connectivity
- A standard API in the `java.sql` package
- The bridge between Java and SQL databases
- Part of the Java platform since JDK 1.1

JDBC architecture



JDBC drivers

- Vendor-specific implementation of the JDBC API
- Added as a Maven dependency
- Registered automatically via `ServiceLoader`
- Old way you see in older apps sometimes: `Class.forName("org.h2.Driver")`
- But that's no longer needed



A note on SQL

P

This module assumes you know SQL

D

If you need a refresher, check the SQL materials

Y

We won’t be teaching SQL syntax here

Follow-along demo: project setup

- Create a Maven project
- Add the H2 database dependency
- Create a `Main` class
- Connect to an H2 file-based database
- Print database name and version

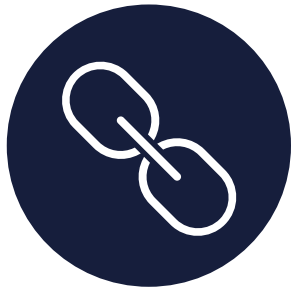
Section 2

Connections & Basic Queries

Setting up a connection

- `DriverManager.getConnection(url, user, password)`
- JDBC URL format: `jdbc:<database-type>:<location>/<schema-name>`
- Common databases: H2 (file/memory), PostgreSQL, MySQL, Oracle
- Each database has its own URL syntax

The three main characters



Connection

The link to the database

Statement

Sends SQL to the database

ResultSet

Holds query results

Resources and the database

- Every connection costs resources
  - Java + database
- A Connection is like a phone call
- Hang up when you’re done
- Use try-with-resources for automatic cleanup

Try-with-resources example

```
try (Connection conn = DriverManager.getConnection(url, user, pass);
     Statement stmt = conn.createStatement()) {
    // use stmt here
} // automatically closed
```

Executing queries

DDL

```
stmt.execute("CREATE TABLE ...")
```

DML

```
stmt.executeUpdate("INSERT ...")
```

Returns affected row count

Query

```
stmt.executeQuery("SELECT ...")
```

Returns `ResultSet`

What is a ResultSet?

- An in-memory representation of query results
- Holds rows and columns returned by a **SELECT** statement
- Starts before the first row (cursor position)
- You iterate through it and read column values using methods

Iterating a ResultSet

- `rs.next()` moves the cursor forward
- Returns `false` when no more rows
- Access columns: `rs.getString("name")`, `rs.getInt("age")`
- You can also access the columns by their index
- Indices start at 1 and not 0
- By name is preferred over by index

ResultSet example

```
try (ResultSet rs = stmt.executeQuery("SELECT * FROM students")) {
    while (rs.next()) {
        String name = rs.getString("name");
        int age = rs.getInt("age");
        System.out.println(name + " (" + age + ")");
    }
}
```

Mini exercise 1

First CRUD operations

- Create a `students` table (id, name, email, age)
- Insert 3 students
- Select and print all students
- Update a student's age
- Delete a student
- Verify results after each step

Section 3
PreparedState
iection

```
String sql = "SELECT * FROM users WHERE name = '" + input + "'";
```

The problem with String concatenation

- What if `input` is: `' OR '1'='1`?
- The database executes something you didn't intend

Why SQL injection is dangerous

- O Leak sensitive data
- l Delete entire tables
- h Bypass authentication
- E One of the OWASP Top 10 vulnerabilities

The solution: PreparedStatement

- Use ? placeholders instead of concatenation
- Set parameters with typed methods
- Database receives query and data separately
- SQL injection becomes impossible

WARNING

Just using PreparedStatement doesn't fix anything, use the ? placeholder instead of concatenation!

PreparedStatement example

```
String sql = "SELECT * FROM users WHERE name = ?";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, name);
    try (ResultSet rs = ps.executeQuery()) {
        // process results
    }
}
```

PreparedStatement vs Statement

	Statement	PreparedStatement
Safety	Vulnerable to injection	Parameterized = safe
Performance	Parsed every time	Cached by database
Readability	Messy concatenation	Clean placeholders

Mini exercise 2

PreparedStatements in practice:

- Experience SQL injection on a vulnerable app
- Replace `Statement` with `PreparedStatement`
- Verify injection no longer works
- Create an extra `findByName(String name)` method

Section 4

Mapping ResultSets to Objects

The problem with ResultSets

- `rs.getString("name")` everywhere
- Column names as strings ⟶ typo = runtime error
- Not type-safe
- Repetitive and error-prone

The solution: map to Java objects

Create a Java class (POJO or record)

Write a mapping method: ResultSet row to object

Work with objects instead of ResultSets

Mapping step by step

e

`mapRow(ResultSet rs)`

Reads columns, returns object

t

`findAll()`

Returns `List<Student>`

`findById(long id)`

Returns `Optional<Student>`

Mapping example

```
private Student mapRow(ResultSet rs) throws SQLException {
    return new Student(
        rs.getLong("id"),
        rs.getString("name"),
        rs.getString("email"),
        rs.getInt("age")
    );
}
```

Patterns emerging

- Every entity: table + columns + Java class
- Same structure, different details
- Lots of boilerplate code
- This is what ORM frameworks automate

The DAO pattern

- Data Access Object
  - Separates data from business logic
- Interface defines the contract
- Implementation uses JDBC
- Rest your code doesn’t need to know about SQL

DAO structure

```
public interface StudentDao {
    List<Student> findAll();
    Optional<Student> findById(long id);
    void save(Student student);
    void update(Student student);
    void delete(long id);
}
```

Mini exercise 3

Student DAO

- Create a `Student` class or record
- Define a `StudentDao` interface
- Implement it with JDBC and PreparedStatements
- Test all CRUD methods from `main`
- **Bonus:** add a `Course` entity with its own DAO
- **Bonus:** add a console app for CRUD operations

Section 5

DataSources & Connection Management

The problem with DriveManager

- Every `getConnection()` creates a new connection
- Connection setup is expensive (handshake, auth, resources)
- Real apps handle hundreds of requests per second

Connection pooling

- A pool of reusable connections
- a Borrow a connection, use it, return it
- N Much faster than creating new ones
- A Controls max concurrent connections

DataSource: the abstraction

- `javax.sql.DataSource` interface
- Replaces `DriverManager` as connection source
- Configurable: pool size, timeout, etc.
- Standard Java API

H2 DataSource setup

```
JdbcDataSource dataSource = new JdbcDataSource();
dataSource.setURL("jdbc:h2:file:./mydb");
dataSource.setUser("sa");
dataSource.setPassword("");
```

Updating your DAO

Inject `DataSource` via constructor

Call `dataSource.getConnection()` in each method

Connection returned to pool after try-with-resources

Updated DAO constructor

```
public class JdbcStudentDao implements StudentDao {
    private final DataSource dataSource;

    public JdbcStudentDao(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    public List<Student> findAll() {
        try (Connection conn = dataSource.getConnection();
            // ... rest of query
```

Mini exercise 4

DriveManager to DataSource

- Create a `DataSource` for your H2 database
- Update `StudentDao` to receive a DataSource
- Verify all CRUD operations still work
- **Bonus:** create a `DatabaseConfig` utility class

Morning wrap-up

- JDBC: the bridge between Java and databases
- Connections, Statements, ResultSets
- Safe queries with PreparedStatements
- Mapping results to Java objects
- DAO pattern for clean data access
- DataSources for connection management

This afternoon: building an ORM Framework

- Build a lightweight ORM on top of JDBC
- Centralize connection management
- Create generic object mapping
- Apply it to an H2 file-based database